FireRed Remake / rust-prototype

Raccolta domande e risposte

Documento generato dalla conversazione con l'agente Cursor sul repository FireRed Remaster / Native Rust / Let's Go Remake.

Contiene le interazioni in ordine cronologico (12 novembre 2026, fuso Europe/Rome), con domanda e sintesi strutturata della risposta.

I canvas interattivi prodotti in chat restano disponibili nei link Cursor; questo PDF è una copia testuale consolidata per archivio e revisione offline.

Indice

1. Analisi tecnica completa (focus rust-prototype)
2. Migliore PR per newcomer + 20 feature / bug / incomplete
3. Marker TODO/FIXME/… e funzioni incomplete
4. Valutazione organizzativa e voto accettazione PR
5. Piano di contributo 30 giorni (Cursor Pro + Claude)
6. La migliore PR nelle prossime 48 ore
7. Richiesta PDF di tutta la conversazione

DOMANDA 1

Analizza completamente questa repository. Non scrivere codice. Voglio un report tecnico dettagliato in italiano con: architettura, dipendenze, stato, parti complete/incomplete, TODO, qualità, problemi architetturali, manutenibilità, opportunità contributor. Focus su rust-prototype. Alla fine tabella difficoltà / impatto / effort / probabilità accettazione PR.

RISPOSTA

Verdetto: il remaster C++ è il prodotto giocabile; rust-prototype è un source-port a metà strada — dati/mondo avanti, battaglia/script early-game, frontend Let's Go già forte ma non misura la completezza meccanica.

Architettura

Tre linee nello stesso repo:
1) FireRed Remaster — C++/gbarecomp (logica GBA riconfermata + UI HD)
2) FireRed Native — libreria Rust senza renderer (port da pret/pokefirered)
3) Let's Go Remake — frontend Bevy 0.17 su Native

In rust-prototype: libreria (lib/core/field/encounter/battle/actors/free) vs binario Bevy (main/presentation/models/field_screen/battle_*). Dati JSON non nel clone pubblico (include_str! + importers).

Stato e metriche

~14.5k LOC Rust; 180 mappe; ~51% comandi script VM; ~11% effetti mossa (24/214); comportamento nuovo spesso UNVERIFIED. Completo: movimento/collisioni, core party/PC/bag/Pokédex, incontri terra, catch/flee/whiteout, status base, math Gen3, slice 3D Pallet/Route 1. Incompleto: weather/screens/Substitute/doubles, trainer multi-mon, HM, water encounters, Safari setup, evolution, UI bag/PC/dex.

Qualità e rischi

Punti di forza: fail-loud, provenance, ~73 test, nomi pret. Debolezze: battle.rs (~3200 LOC) e field_screen.rs (~1900) come god-object; onboarding alto senza ROM; dualità C++/Rust. Manutenibilità media-condizionata.

Tabella opportunità (sintesi)

Più accettabili: ScrCmd piccoli, una famiglia effetto mossa source-backed, docs coverage VM, fixture CI sintetiche. Bassa probabilità: redesign inventati. Canvas prodotto: Report tecnico rust-prototype.

DOMANDA 2

Se fossi un maintainer e dovessi accettare una PR da un nuovo sviluppatore, quale sarebbe la migliore? Analizza: 20 feature semplici mancanti, bug facili, parti incomplete. Ordina dal facile al difficile. Per ogni punto: file, LOC, complessità, rischio regressioni, valore.

RISPOSTA

Migliore PR da newcomer: guida first-PR + matrice ScrCmd/Special portati vs mancanti (solo docs/tooling). Zero regressioni, zero conflitto ownership Codex, massimo valore triage.

Se vuole codice runtime al primo colpo

HUD contestuale + tabella musica per mappa (main.rs, music.rs, ~40–80 LOC): oggi con 180 mappe l'help dice ancora “North: Route 1…” e quasi tutto suona Pallet.

Top facili

- Coverage VM + CONTRIBUTING — docs/* — 80–200 LOC — rischio nullo — valore altissimo
- HUD per mappa/connections — main.rs — 25–60 — rischio molto basso
- Music map table — music.rs — 20–50
- Più glifi in font_safe_text — main.rs — 10–25
- Special no-op whitelist source-backed — field_screen.rs — 15–40 — chiedere ownership

Bug facili / incomplete

Bug: HUD Route1 ovunque; musica Pallet; accenti incompleti; onboarding senza JSON; ShowMonPic/Door silenziosi; Fame Checker abort smoke; Struggle assente; UI ball fissa.

Incomplete strutturali: VM ~51%; effetti ~11%; trainer multi-mon; water; actors wandering; UI bag/PC/dex; evolution/HM/Safari; diorama Viridian; parity C differita.

Canvas: PR ideale e backlog facile.

DOMANDA 3

Trova tutte le occorrenze di TODO, FIXME, BUG, HACK, TEMP, NOT IMPLEMENTED, UNIMPLEMENTED. Analizza anche le funzioni incomplete. Report con percorso, problema, soluzione suggerita, difficoltà.

RISPOSTA

Risultato chiave: in rust-prototype e fr_*.cpp quasi non ci sono TODO/FIXME/HACK. Il debito è espresso come Unported/Unsupported/deferred e in docs/STATE.md. Zero todo!/unimplemented! in Rust. FIXME nel remaster/rust: 0. TEMP come marker: 0.

Marker letterali first-party (estratto)

- docs/STATE.md:784 — TODO mouse glitchy, tileset 3D, HD Pokédex Area/Size — Media
- rust-prototype/README.md:92 — battle audio not implemented — Bassa
- gbarecomp multiplayer_config.cpp — multiboot / cross-version not implemented — Molto alta
- GBA_ACCURACY_BURNDOWN.md — Mosaic + bitmap 3/4/5 — Alta
- ROADMAP.md — LDM/STM S-bit still TODO — Alta
- recomp_launcher.h — DualShock multitap “hack” opt-in — Bassa

Funzioni incomplete rilevanti

- field.rs Vm::command — catch-all Unported (~metà ScrCmd)
- field_screen.rs — Special / Door / ShowMonPic / Sound no-op o Err
- battle.rs — effetti + Struggle + AI opcode
- actors.rs / encounter.rs / core.rs — deferred
- music.rs / main.rs hud — incompleti UX
- fr_diorama.cpp — tileset oltre Pallet

Nota: HP underflow in core.rs è bug FireRed preservato di proposito — non correggerlo. Esclusi imgui/stb (migliaia di FIXME upstream). Canvas: Report marker TODO e incomplete.

DOMANDA 4

Analizza la repository dal punto di vista organizzativo: personale o collaborativo? Strutturato per contributor? Convenzioni di coding? Punti dove una PR è facile da revisionare? Voto 1–10 probabilità che il maintainer accetti una PR ben fatta.

RISPOSTA

È un progetto personale di Ben con collaborazione strumentale (soprattutto agenti AI), non una community OSS classica.

Dettaglio

- Personale+: AGENTS.md lo dice; un autore git; deploy sulla macchina di Ben
- Collaborazione reale: Codex ↔ Gemini con ownership file (RUST-COLLABORATION.md)
- Pronto per contributor solo in parte: no CONTRIBUTING/template/CI amichevole; onboarding alto
- Convenzioni sì (C++17, toggle settings, fail-loud, source-backed) ma poco automatizzate
- PR facili da review: docs/coverage, HUD/music, fr_diorama+check_diorama, un ScrCmd

Voto

7/10 per una PR ben fatta e allineata. Condizionato: docs/polish ≈ 8–9; ScrCmd fedele ≈ 7; battle.rs non concordato ≈ 3–4. Canvas: Valutazione organizzativa contributo.

DOMANDA 5

Agisci come senior Rust developer. Ho Cursor Pro e Claude. Voglio contribuire entro 30 giorni. Piano settimanale: cosa studiare, file, sistemi, prima/seconda PR, competenze Rust. Ottimizza per accettazione PR.

RISPOSTA

Strategia: 4–6 PR piccole; nelle prime 2 settimane evita battle.rs / field_screen.rs.

Settimana 1 — Orientamento

Studiare architettura e ownership. Leggere README, AGENTS, handoff, collaboration, Cargo.toml, lib.rs, main Preferences. PR1: docs CONTRIBUTING-RUST + matrice ScrCmd/Special (~8–9/10). PR2: README aspettative 180 mappe ≠ story. Rust: moduli, Result, feature Cargo.

Settimana 2 — Runtime sicuro

PR1: HUD contestuale. PR2: tabella temi musica (senza nuovi WAV). File: main.rs, music.rs, lib.rs, smoke.rs. Rust: helper puri + test.

Settimana 3 — Adapter

PR1: ShowMonPic/Door feedback OPPURE font_safe_text se ownership bloccata. PR2: 2–5 special no-op source-backed (chiedere ownership). File: field.rs, field_screen.rs.

Settimana 4 — Prima meccanica piccola

PR1: un ScrCmd OPPURE water encounter (mirror land). PR2: aggiorna matrice coverage + test. Evitare: weather, switching, rewrite battle, commit asset. Canvas: Piano contributo 30 giorni.

DOMANDA 6

Dimentica tutto il resto. Qual è la migliore PR che potrei aprire nelle prossime 48 ore? Utile, piccola, facilmente revisionabile, compatibile con l'architettura, alte probabilità di accettazione. Mostra esattamente quali file modificare.

RISPOSTA

PR migliore: documentazione — guida first-PR + matrice ScrCmd/Special portati vs mancanti. Utile per onboarding/triage, ~100–200 righe docs, zero rischio runtime, nessun conflitto ownership Codex, probabilità ~8–9/10.

File da toccare

- CREARE: docs/CONTRIBUTING-RUST.md
- CREARE: docs/RUST-VM-COVERAGE.md
- MODIFICARE (1–3 link): rust-prototype/README.md
- OPZIONALE (1 riga): docs/STATE.md o AGENTS.md

NON toccare: src/**, importers, JSON/assets, battle.rs, field_screen.rs.

Contenuto minimo: regole Ben; tabella comandi/special (portato / no-op / mancante); link Contributing nel README. Alternativa codice: solo rust-prototype/src/main.rs (HUD contestuale). Per 48 ore e massima accettazione resta meglio la PR docs.

DOMANDA 7

Dammi un PDF di tutte le risposte date fino ad ora con relative domande.

RISPOSTA

Questo documento è la risposta: PDF consolidato con domanda e risposta per ogni turno della conversazione.

Percorso file:
/opt/cursor/artifacts/report-conversazione-firered-rust.pdf

Canvas prodotti in conversazione

1. Report tecnico rust-prototype
2. PR ideale e backlog facile
3. Report marker TODO e incomplete
4. Valutazione organizzativa contributo
5. Piano contributo 30 giorni

Nota: le risposte originali includevano canvas interattivi e tabelle estese. Questo PDF ne riporta il contenuto essenziale in forma testuale archiviabile. Nessun codice di gioco è stato modificato per produrre questo file.